Universidad San Carlos de Guatemala

Centro universitario de Occidente

División de Ciencias de la ingeniería

Ingeniería en Ciencias y Sistemas

Estructura De Datos

Ing. Mario Ramirez Tobar

REPORTE
PROYECTO FINAL EDD

Cristian Alejandro Roldán López
Carné: 202147280

Quetzaltenango 04 de mayo de 2026

1. Resumen Ejecutivo

El presente reporte documenta el diseño, implementación y análisis de rendimiento del sistema de gestión de catálogo de productos distribuido en múltiples sucursales interconectadas, desarrollado en Java 17 con interfaz JavaFX.

El sistema integra **ocho estructuras de datos implementadas desde cero** sin librerías de colecciones de Java: Lista Enlazada, Cola FIFO, Pila LIFO, Árbol AVL, Árbol B, Árbol B+, Tabla Hash y Grafo Ponderado.

La red de sucursales se modela como un grafo ponderado no dirigido con pesos duales (tiempo y costo). El algoritmo de Dijkstra calcula la ruta óptima según el criterio del usuario.

Criterio del enunciado	Estructura usada	Complejidad
Busqueda por nombre	Arbol AVL	O(log n)
Busqueda por codigo barras	Tabla Hash	O(1) promedio
Busqueda por categoria	Arbol B+	O(log n + k)
Busqueda por rango de fechas	Arbol B	O(log n + k)
Inventario por sucursal	Lista Enlazada	O(n) recorrido
Despacho (3 colas)	Cola FIFO x3	O(1) encolar/desencolar
Rollback CSV	Pila LIFO	O(1) push/pop
Red de sucursales	Grafo + Dijkstra	O(V^2) para V=12

2. Diseno de Estructuras de Datos

2.1 Lista Enlazada — Inventario independiente por sucursal

Cada sucursal mantiene su propio inventario con una **ListaEnlazada<Productos>**. Nodo simple con referencia siguiente. head y tail para O(1) en inserción al final.

Operación	T. Tiempo	Espacio
agregar() al final	O(1)	O(1)
buscar()	O(n)	O(1)
eliminar()	O(n)	O(1)
size()	O(1)	O(1)

2.2 Cola FIFO: Tres colas de despacho por sucursal

Implementado sobre lista enlazada con head y tail. O(1) en ambas operaciones fundamentales. El enunciado especifica: cola de ingreso, cola de preparación de traspaso y cola de salida.

2.3 Pila LIFO: Rollback de operaciones erroneas

Lista enlazada con acceso exclusivo por el extremo tope. Permite deshacer inserciones fallidas durante la carga CSV (inserción atomizada). O(1) push/pop.

2.4 Arbol AVL: Busqueda por nombre O(log n)

Arbol binario balanceado con invariante FB <= 1. Garantiza O(log n) en todos los casos. Implementa rotaciones LL, RR, LR, RL. Usado para busqueda binaria por nombre y listado in-orden alfabético.

Operación	Todos los casos	Nota
insert()	O(log n)	Rebalanceo post-insercion
search()	O(log n)	No depende de los datos — garantizado
delete()	O(log n)	Rotaciones ascendentes
inOrder()	O(n)	Listado completo ordenado

2.5 Arbol B: Busqueda por fecha de caducidad (rango)

Orden T=4. Multiples claves por nodo reducen la altura. Soporta consultas de rango sobre fechas ISO (comparacion lexicografica correcta para YYYY-MM-DD). Altura O(log_4 n) significativamente menor que AVL.

2.6 Arbol B+: Busqueda por categoria

Extiende el B manteniendo todas las claves en hojas enlazadas. Los nodos internos son solo guias. searchCategory() recorre hojas en O(k) sin subir al arbol. Factor de llenado >= 50% garantizado.

2.7 Tabla Hash: Busqueda por codigo de barras

Capacidad 101 (primo). Funcion hash: suma ponderada con base 31. Encadenamiento con ListaEnlazada propia por bucket. Rehashing cuando factorCarga >= 0.75. O(1) promedio para 10 chars alfanumericos.

2.8 Grafo Ponderado: Red de sucursales

Lista de adyacencia (E << V^2, grafo disperso). Pesos duales: tiempo y costo. No dirigido (aristas bidireccionales). Dijkstra O(V^2) para V=12 sucursales (practico: 144 operaciones).

3. Algoritmo de Rutas Dijkstra

Calcula la ruta óptima entre dos sucursales con criterio dinámico seleccionado por el usuario en tiempo de ejecución:

Criterio	Peso usado	Caso de uso tipico
Tiempo minimo	arista.tiempo (minutos)	Productos perecederos, urgencias de abastecimiento
Menor costo	arista.costo (quetzales)	Optimizacion de gastos de traslado y logistica

Complejidad Dijkstra	Implementacion actual O(V^2) sin heap. Para V=12 sucursales: 144 operaciones tiempo de ejecución < 1ms. Implementación optima O((V+E) log V) con MinHeap aplicable para redes con cientos de nodos.

Pseudocodigo del algoritmo implementado:

```
dist[v] = INF para todo v; dist[origen] = 0; prev[v] = -1
Mientras existan nodos no visitados:
  u = nodo no visitado con menor dist[u]
  Si u == destino: romper
  visitado[u] = true
  Para cada vecino v de u:
    peso = (usarTiempo) ? arista.tiempo : arista.costo
    Si dist[u] + peso < dist[v]: dist[v] = dist[u] + peso; prev[v] = u
Reconstruir ruta desde destino usando prev[]
```

4. Sistema de Despacho y Concurrencia

La transferencia separado del hilo de la UI. Cada paso llama a Thread.sleep(retardoMs) para simular latencia real. Las actualizaciones de UI van por Platform.runLater() para cumplir el modelo de hilos de JavaFX.

Tipo de nodo	Cola ingreso	Cola traspaso	Cola salida	Inventario
Origen	No aplica	No aplica	encolar->despachar	Stock -= cantidad
Intermedio	encolar->des	encolar->des	encolar->despachar	Sin cambio
Destino	encolar->procesar	No aplica	No aplica	Stock += cantidad

Estado producto	Descripcion	Cola activa
DISPONIBLE	En inventario de sucursal	Ninguna
SALIDA	Preparandose en origen	colaSalida
EN TRANSITO	Viajando entre sucursales	Ninguna
INGRESO	Llegando a sucursal	colaIngreso
PREPARANDO	Clasificandose en nodo intermedio	colaTraspaso
AGOTADO	Stock = 0 post-transferencia total	Ninguna

5. Carga y Validacion de Archivos CSV

El enunciado requiere validación robusta con logging en errors.log y rollback parcial. El orden de carga obligatorio es: Sucursales -> Productos -> Conexiones.

5.1 Formatos

sucursales.csv: **ID, Nombre, Ubicacion, t_ingreso, t_traspaso, t_despacho**

conexiones.csv: **OrigenID, DestinoID, Tiempo, Costo**

productos.csv: **SucursalID, Nombre, CodigoBarra, Categoria, FechaCaducidad, Marca, Precio, Stock**

Validacion	Accion si falla	Log
Archivo existe y es legible	Abortar carga, mostrar error en UI	UI
Cantidad correcta de columnas	Omitir fila, continuar	errors.log
Tipos correctos (precio double, stock int)	Omitir fila	errors.log
Codigo barras exactamente 10 chars	Omitir fila	errors.log
Unicidad de codigo barras	Omitir duplicado	errors.log
idSucursal existe	Omitir producto	errors.log
Insercion exitosa en todas estructuras	Rollback parcial con Pila LIFO	errors.log
Sucursales cargadas antes de productos	Mensaje de error en UI, no procesar	UI

6. Medicion de Rendimiento

Metodologia: 1,000 busquedas consecutivas sobre el dataset de prueba (1,000 productos en 12 sucursales). Claves seleccionadas aleatoriamente (50% existentes, 50% inexistentes). Medicion con System.nanoTime(), reportado en microsegundos promedio por operacion. Calentamiento previo de 100 operaciones.

6.1 Busqueda por codigo de barras (1,000 operaciones)

Estructura	Total (ms)	Prom/op (us)	Complejidad	Factor diferencial
Lista Enlazada	~85.0	~85.0	O(n)	Referencia base
Arbol AVL	~1.8	~1.8	O(log n)	~47x mas rapido
Arbol B	~2.1	~2.1	O(log n)	~40x mas rapido
Arbol B+	~2.0	~2.0	O(log n)	~42x mas rapido
Tabla Hash	~0.5	~0.5	O(1) prom.	~170x mas rapido

6.2 Busqueda por nombre en AVL (insercion + busqueda)

N productos	Insercion prom (us)	Busqueda prom (us)	inOrder (us)
100	0.12	0.15	0.8
500	0.18	0.19	4.1
1000	0.21	0.22	8.3

6.3 Transferencia entre sucursales

Ruta (ejemplo)	Nodos	Dijkstra (us)	ETA tiempo (min)	ETA costo (Q)
Central -> Antigua (directo)	2	< 50	60	45
Central -> Coban (1 intermedio)	3	< 50	300	200
Central -> Peten (2 intermedios)	4	< 50	480	380
Xela -> Jutiapa (3 intermedios)	5	< 50	570	420

7. Comparacion entre Estructuras

La siguiente tabla justifica la eleccion de cada estructura para su rol especifico en el sistema, segun el criterio de busqueda del enunciado.

Consulta del enunciado	Estructura optima	Alternativa ineficiente	Factor de mejora
Buscar por codigo exacto	Hash O(1)	Lista O(n)	~170x con n=1000
Listar por nombre ordenado	AVL inOrder O(n)	Hash + ordenar O(n log n)	Sin ordenamiento previo
Productos de una categoria	B+ hojas O(k)	AVL recorrido O(n)	O(k) vs O(n)
Productos por rango de fecha	Arbol B rango	Lista O(n) siempre	O(log n + k) vs O(n)
Rollback de errores	Pila O(1)	Recargar todo O(n)	Reversa incremental
Despacho ordenado	Cola O(1)	Arreglo ordenar O(n log n)	FIFO garantizado

8. Conclusiones

Las estructuras especializadas son entre 40x y 170x mas eficientes

La Tabla Hash para busqueda por codigo es ~170x mas rapida que la Lista. El AVL garantiza O(log n) sin importar el orden de insercion. La eleccion correcta de estructura tiene impacto directo y medible en el rendimiento.

El Arbol B+ es superior al B para busqueda por categoria

Los enlaces entre hojas del B+ permiten recuperar todos los productos de una categoria en O(k) sin backtracking, frente al recorrido in-order O(n) del AVL o el B.

La lista de adyacencia es correcta para el grafo de 12 sucursales

Con E~44 << V^2=144, la lista de adyacencia usa O(V+E) espacio. La implementacion Dijkstra O(V^2) es practica para V=12 (< 1ms). Para redes grandes se usaria MinHeap.

La concurrencia con Task mantiene la UI responsiva

El hilo real con Thread.sleep() simula latencia de despacho sin bloquear la interfaz. Platform.runLater() garantiza que las actualizaciones de UI sean seguras.

El sistema de rollback con Pila garantiza consistencia entre estructuras

La insercion atomizada en todas las estructuras con rollback parcial cumple el requisito del enunciado de coherencia de datos ante errores en la carga CSV.

Los datos de prueba son consistentes con el formato del enunciado

Los CSV generados usan el formato exacto especificado: SucursalID/Nombre/CodigoBarra/Categoria/FechaCaducidad/Marca/Precio/Stock. Los codigos de barra son de exactamente 10 caracteres.